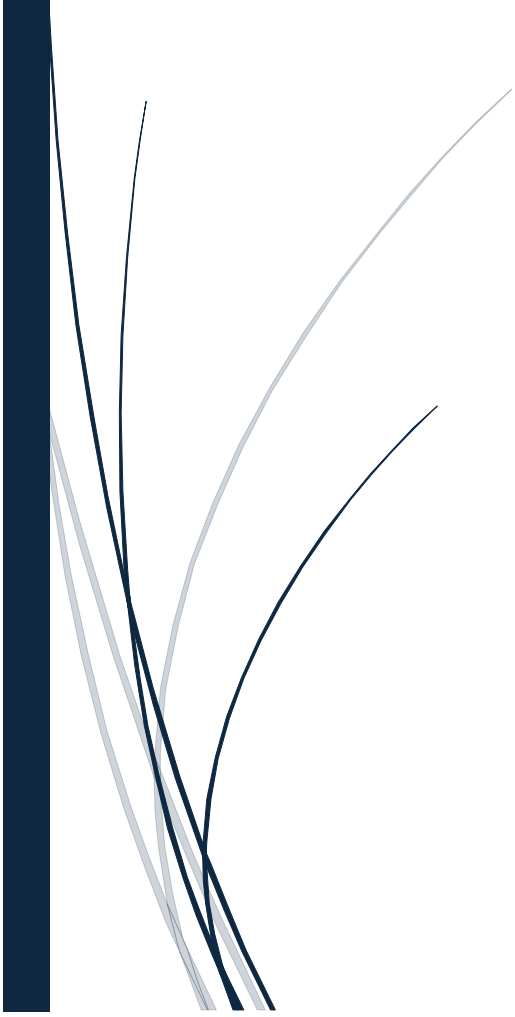
26-5-2024

CoffeeClicker-3

S2_Projct_B_Coffee-Clicker-3



Aleix Batchelli I Abad
Alexandre Contreras i Urbita
Laura Cuenca Plata
Nil de Creus Massot
Armand López Sanmartín

INDEX

Project specification summary 2
Graphical User Interface design 3
Class diagram 13
Development methodology 22
Time costs 23
Conclusions 24
Bibliography 25

INDEX

Project specification summary

Aquest projecte esta inspirat en els jocs IDLE, el qual es basa en dissenyar i implementar un joc similar al Cookie Clicker, però ambientat en cafès o en alguna altre temàtica a escollir que en el nostre cas ens vam basar en crèdits universitaris.

Inicialment, el joc llegeix un arxiu anomenat “config.json”, el qual inclou els diferents paràmetres per a la connexió a la base de dades y per a la connexió amb el servidor.

Una vegada carregat aquest fitxer, els usuaris es poden tant registrar com accedir, en el cas que ja tinguin un compte existent. Per a ambdues funcionalitats els usuraris hauran d’incloure el correu electrònic, i la contrasenya. Una vegada introduïts el programa validarà aquestes dades i segons si han passat les validacions o no, l’usuari en concret, podrà accedir al joc o no. Cal destacar que quan l’usuari es registra per primer cop , el programa, també li demanarà el seu nom d’usuari, el qual haurà de ser únic i cosa que fa que ningun altre usuari podrà tenir el mateix.

Un cop autenticat, l'usuari pot iniciar una nova partida, la qual pot continuar en futures sessions si no ha estat finalitzada. L'objectiu del joc és generar la major quantitat de recursos (crèdits) en el menor temps possible. Els generadors de recursos i millores estan disponibles per incrementar la producció de manera automàtica. En el nostre cas tant aquest generadors com les millores estan ambientades a crèdits i no a cafès.

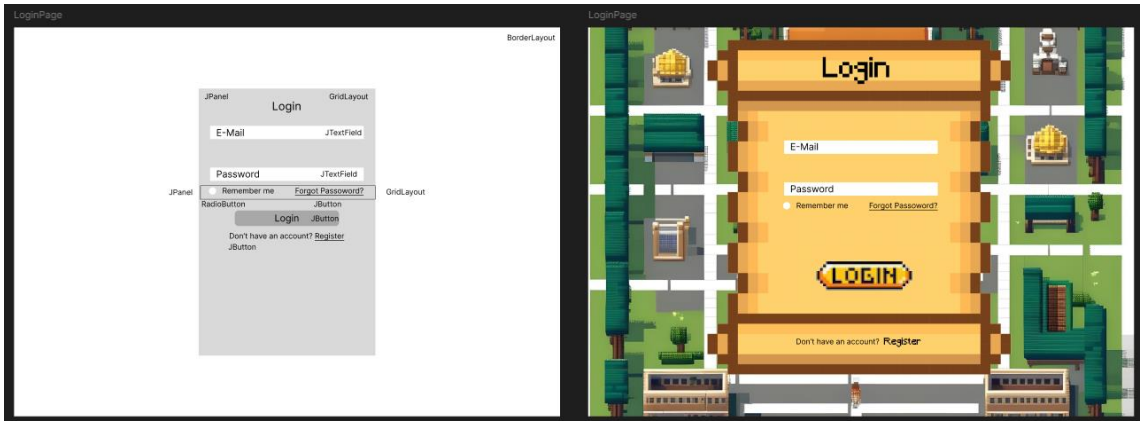
Una vegada escollit si vol crear un a nova partida o seguir un a d’existent, s’hi trobarà comptador de crèdits, un botó per generar crèdits manualment, una botiga de generadors i millores i una taula de generadors disponibles que s'actualitza en temps real. Apart cal destacar que l’usuari també tindrà l’opció tant fer logout com eliminar el seu compte i tota aquella informació que faig referència a ell serà borada de la base de dades.

Finalment, es mostra una gràfica de línies amb l’evolució de les partides finalitzades, permetent seleccionar l’usuari i la partida de la qual es volen veure les estadístiques.

Graphical User Interface design

Pel nostre projecte vam fer un disseny de la GUI utilitzant l'eina Figma. A partir d'aquí vam dividir el projecte en 11 pantalles inicialment que s'han acabat convertint en 7.

Començant per la pantalla de "Login", a continuació podem veure l'exemple de disseny:



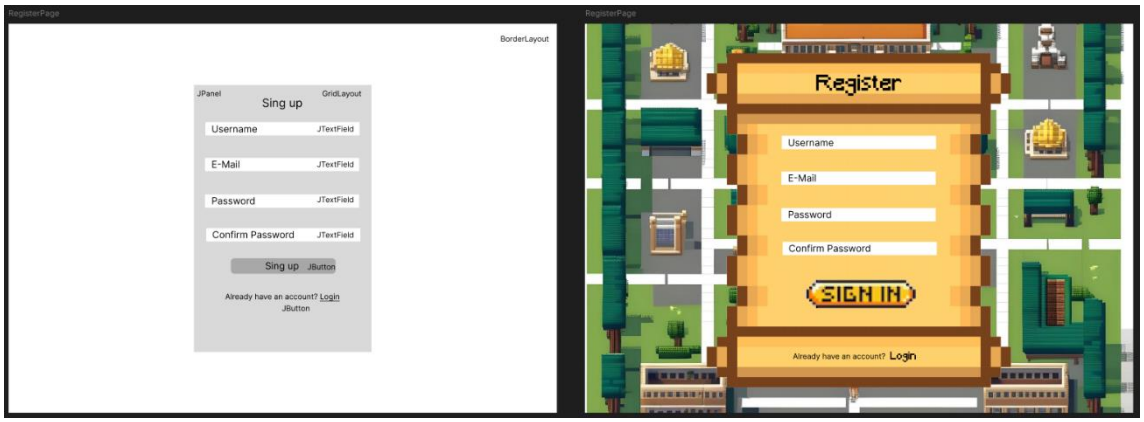
A continuació es pot veure l'exemple final implementat:



Aquesta view està basada en "JLayeredPanel" el qual a partir d'un "Overlay Layout" et permet afegir components en diferentes capes. En la capa 0 tenim una imatge. Les imatges funcionen amb un JImagePanel. Aquesta classe ha estat implementada per nosaltres fent "Override" a la funció "paintComponent" amb opcions a afegir un canal alfa i canviar la resolució per a que ajusti la imatge a la mida del panell o ajustar la llargada o l'alzada de la imatge a la mida del panell que el conté.

La interfície en si està implementada amb un “JPanel” amb un “GridBagLayout” ja que aquest és l’unic que permet centrar els compoents. Finalment els botons han estat implementats amb un “JTexturedButton”, una altra classe de creació propia que fa “Override” a la funció “paintComponent” variant la imatge que pinta depenent de si esà apretat o no. La part del “Remember me” i “Forgot password” les hem tret ja que finalment no les hem implementat.

A continuació la pantalla de Registre:



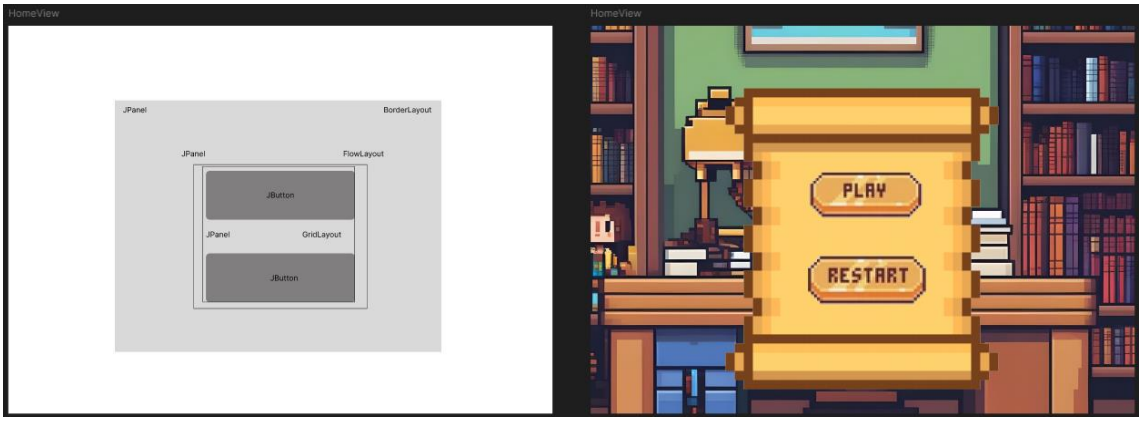
Disseny en Figma.



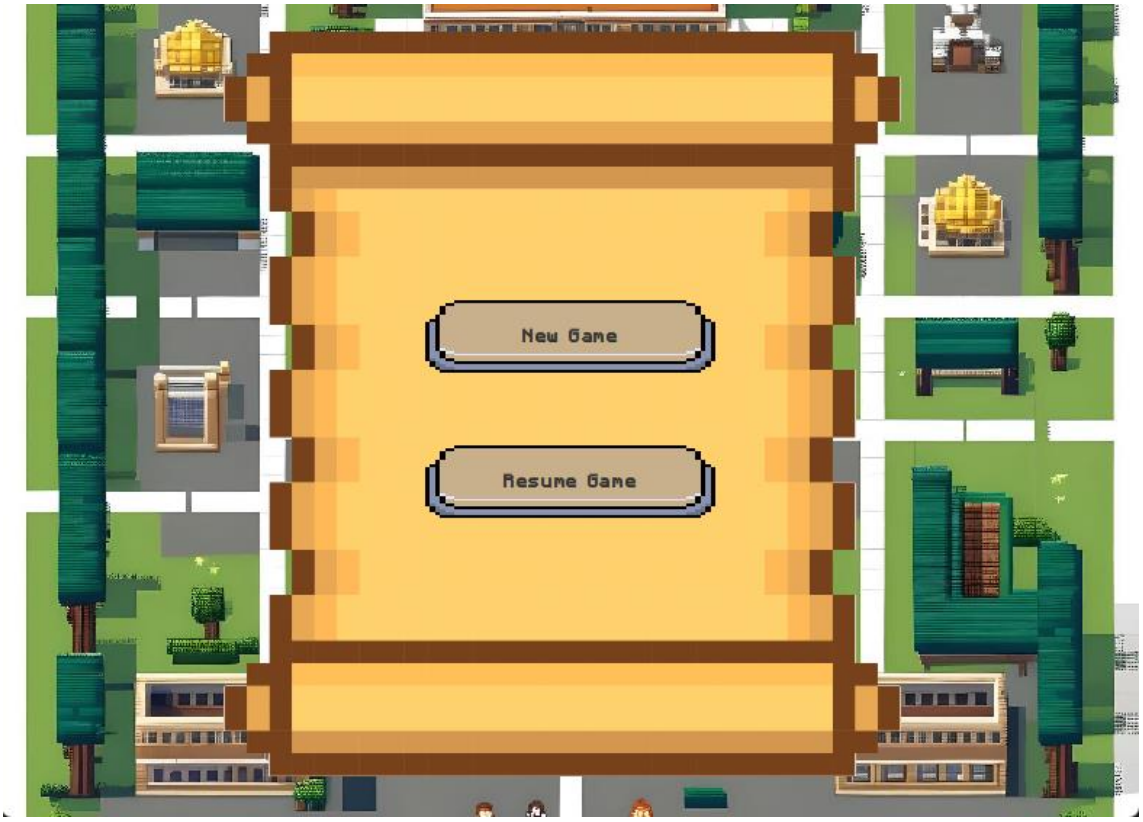
Implementació en Java Swing.

En aquest cas funciona molt similar a la pantalla anterior. Aquesta vista consta de un “JLayeredPane” amb tres capes, la 0 i la 1 per les imatges i la 2 per a la interfície en si. També està implementada amb un “GridBagLayout”.

HomePanel:



Implementació amb figma.



Implementació en Java Swing.

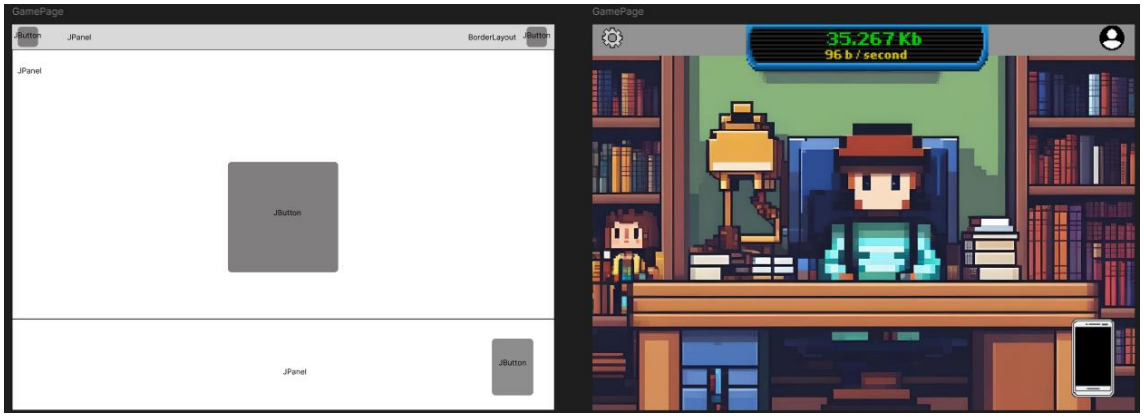
Aquesta classe és molt similar a les anteriors també ja que consta d'un "JLayeredPane" amb tres capes. La GUI com a tal és un "JPanel" amb un "BorderLayout" i en el centre d'aquest panell hi ha un altre "JPanel" amb un "GridLayout" de 2x1 per als dos butons amb una separació vertiacal de 50.

Una petita diferència amb el disseny de figma és que hem acabat optant per mostrar els jocs per fer resume en un "JDialog" que es veu de la següent forma:



Utilitzant un "JScrollPane" i un panell amb un "BoxLayout" en vertical.

GamePanel:



Disseny en Figma.



Implementació amb Java Swing.

Aquest panell torna a ser un “JLayeredPane” però en aquest cas té varies capes:

La capa de background on hi ha la imatge de fons.

La capa noraml on hi ha tota la UI que es veu en pantalla.

La capa de config que és on està el panell flotant per quan es prem el botó de “config”.

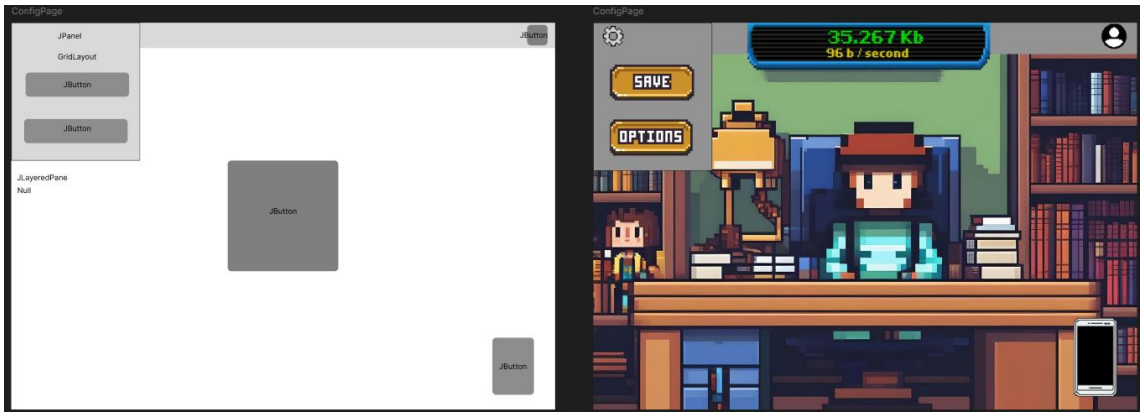
La capa de les botigues on estè el panell de botigues tant de generadors com de millores.

El botó de game el vam treure ja que en la rúbrica posava explícitament que el botó de “logout” ha de ser visible en tot moment des de que s’entra al joc.

La UI en si està basada en un “BorderLayout” amb un panell al Nort, un botó al centre i un panell al Sud.

El panell nord està divdit en 3 utilitzant un “GridLayout” de 1x3. A l’esquerra hi ha un panell amb un “FlowLayout” fixant a l’esquerra, a la dreta el mateix però fixant a la dreta i al centre hi ha un “JLayeredPane” per tal de posar la imatge amb un JLabel davant.

GamePanel – Config:



Disseny amb figma.

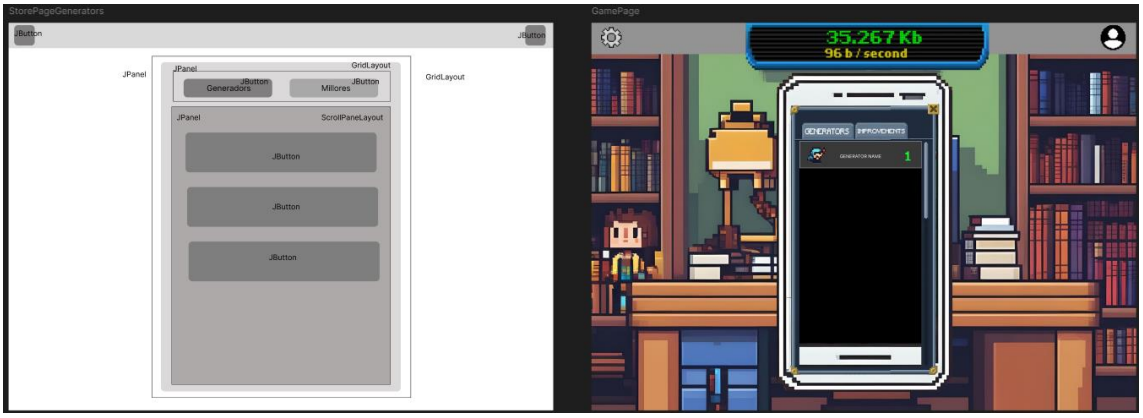


Implementació amb Java Swing.

Aquest és un panell amb un “GridLayout” de 2x1 i a cada casella hi haun panell amb un “FlowLayout”. El primer fixant a l’esquerra pel botó i el segon a la dreta pels altres dos butons.

Vam canviar el text dels botons ja que “options” al final només et deixava anar a la pantalla d’estadístiques i a “save” li vam canviar el funcionament podent fer “exit” en qualsevol moment i el programa et demanarà si vols acabar o guardar la partida.

GamePanel – Stores:



Dissny amb figma.



Implementació amb Java Swing.

En aqueta vista trobem un “JLayeredPane” de dues capes. En la 0 trobem la imatge del teléfon i en la 1 trobem la interfície com a tal.

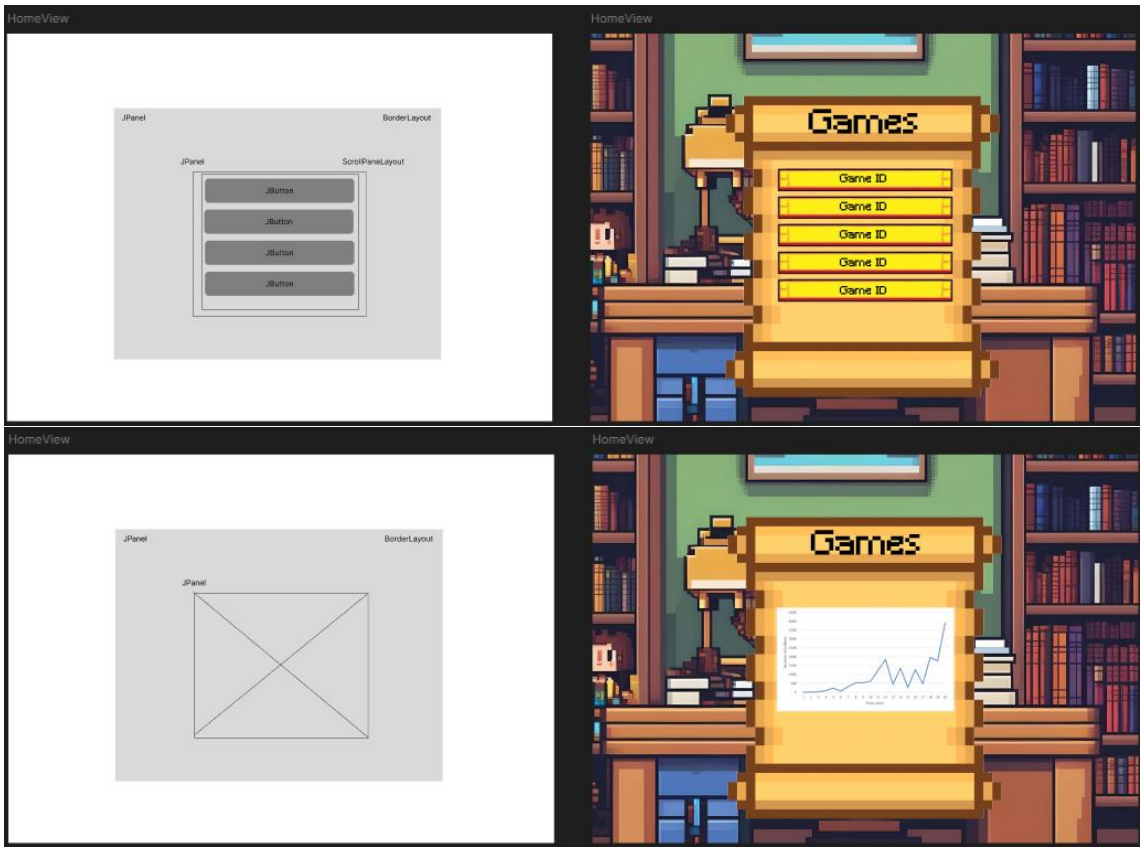
La interfície està feta amb una Panell que de “layout” nul. Aquest conté un altre panell que està col·locat específicament en la posició correcta per semblar la pantalla del telèfon. Aquest panell té un “BorderLayout” amb un panell al nord pels botons i un al centre per a un panell amb un “CardLayout” amb dos panells que utilitzen un “ScrollPane”.

Dins del ScrollPane trobem una JTable amb la informació dels diferents generadors i millores depenent de quin estiguem mostrant.

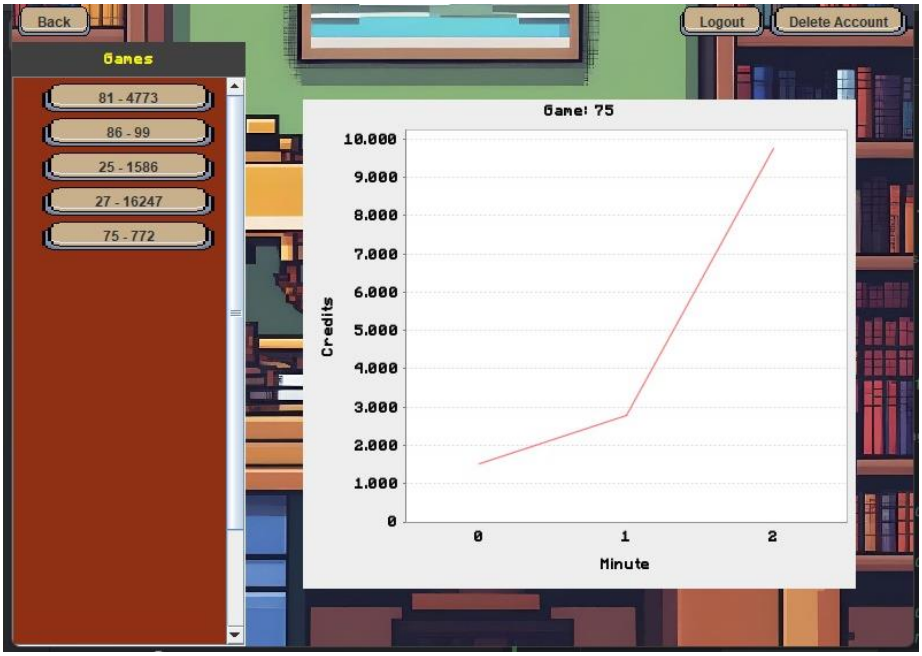


Exemple de millores.

StatisticsPanel:



Disseny amb Figma



Implementació amb Java Swing.

Com es pot apreciar, en el nostre disseny teniem pansat fer dues vistes: una per mostrar els jocs acabats i una per mostrar el gràfic del joc peró finalment vam pensar que era més dinàmic tenir els jocs a un costat i anar actualitzant el gràfic per cada joc.

Per a fer el gràfic hem utilitzat la llibreria “JFreeChart”. La vista consta d’un “JLayeredPane” amb una foto i un panell al davant amb els components . El panell utilitza un “BorderLayout” amb un panell al Nord, un al Oest i un al centre.

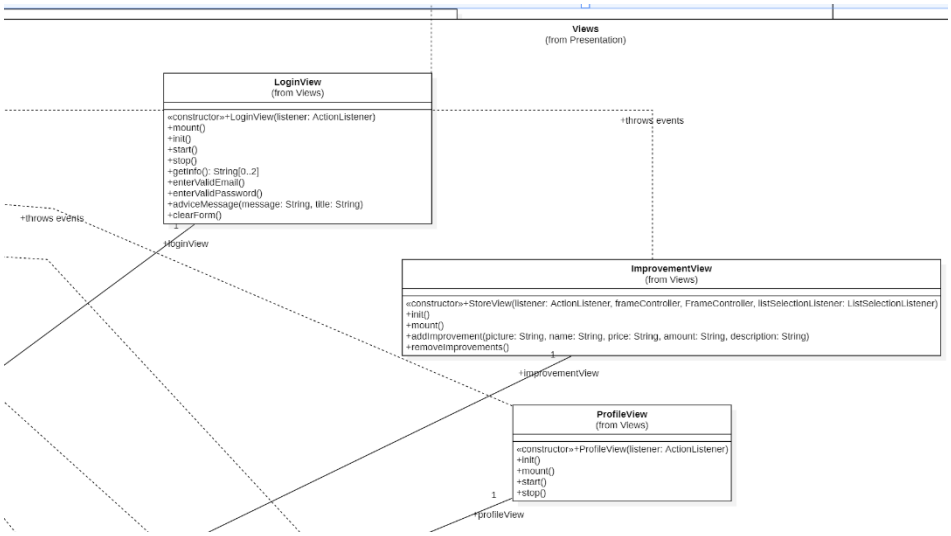
Al nord podem trobar un panell amb un “GridLayout” de 1x2 amb un panell a cada casella amb un “FlowLayout”. El de l’esquerra fixant els components a l’esquerra i el de la dreta fixant-los a la dreta.

Al Oest trobem un panell amb un “BorderLayout” amb una “JLabel” al nord i un “ScrollPane” al centre el qual conté un panell amb un “BoxLayout” amb alineació vertical per als botons dels jocs.

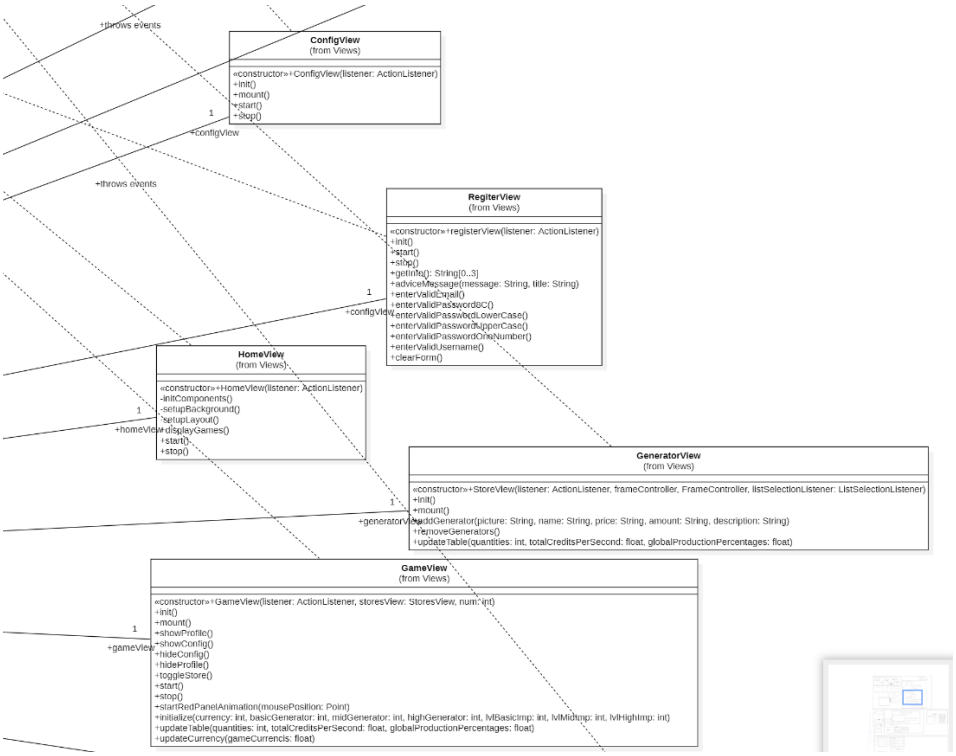
Al centre trobem un “ChartPanel” el qual és un component de la llibreria ja esmentada anteriorment. Per a fer-lo més petit hem afegit un “Border” que actua com a “padding” per a centrar el panell i que no ocupi tot l’espai del qual disposa.

Class diagram

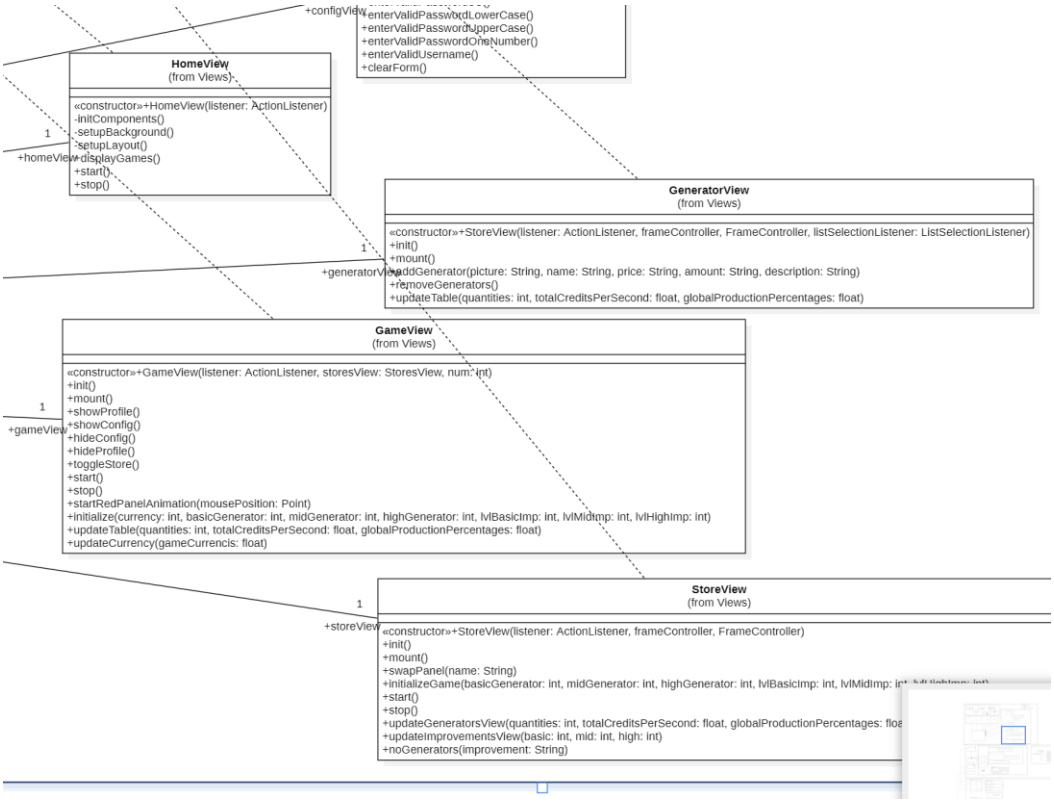
Per realitzar l'estructuració del programa CoffeeClicker, hem seguit els estàndards de la programació orientada a objectes i hem dividit el projecte en tres capes principals: la capa de presentació, la capa de negocis i la capa de persistència.



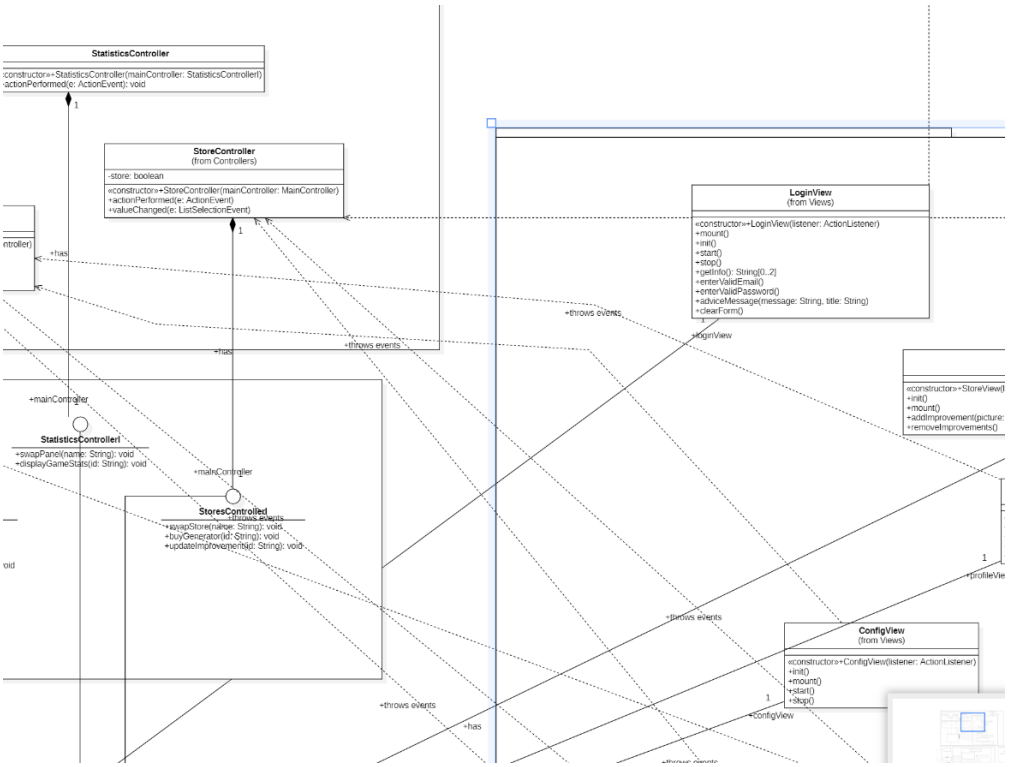
Il·lustració 1- Capa Presentation | Views



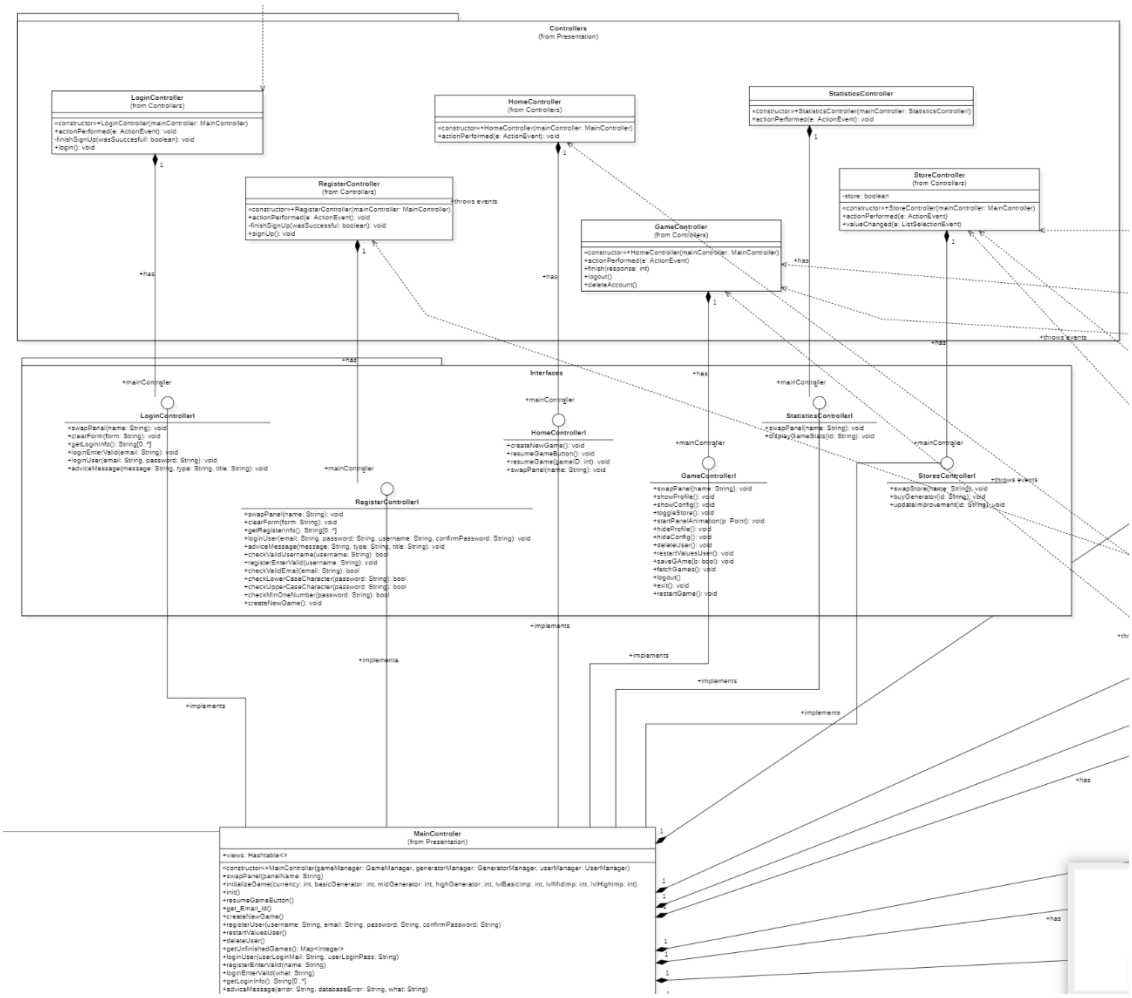
Il·lustració 2 - Capa Presentation | Views



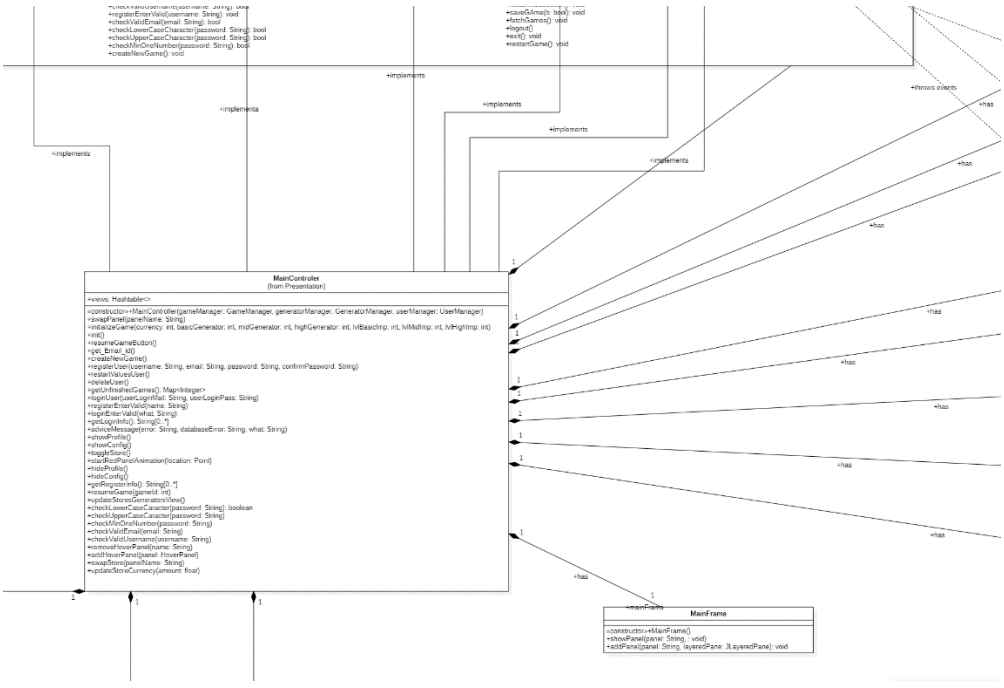
Il·lustració 3 - Capa Presentation | Views



Il·lustració 4 - Capa Presentation | Event throwing from Views to Controllers

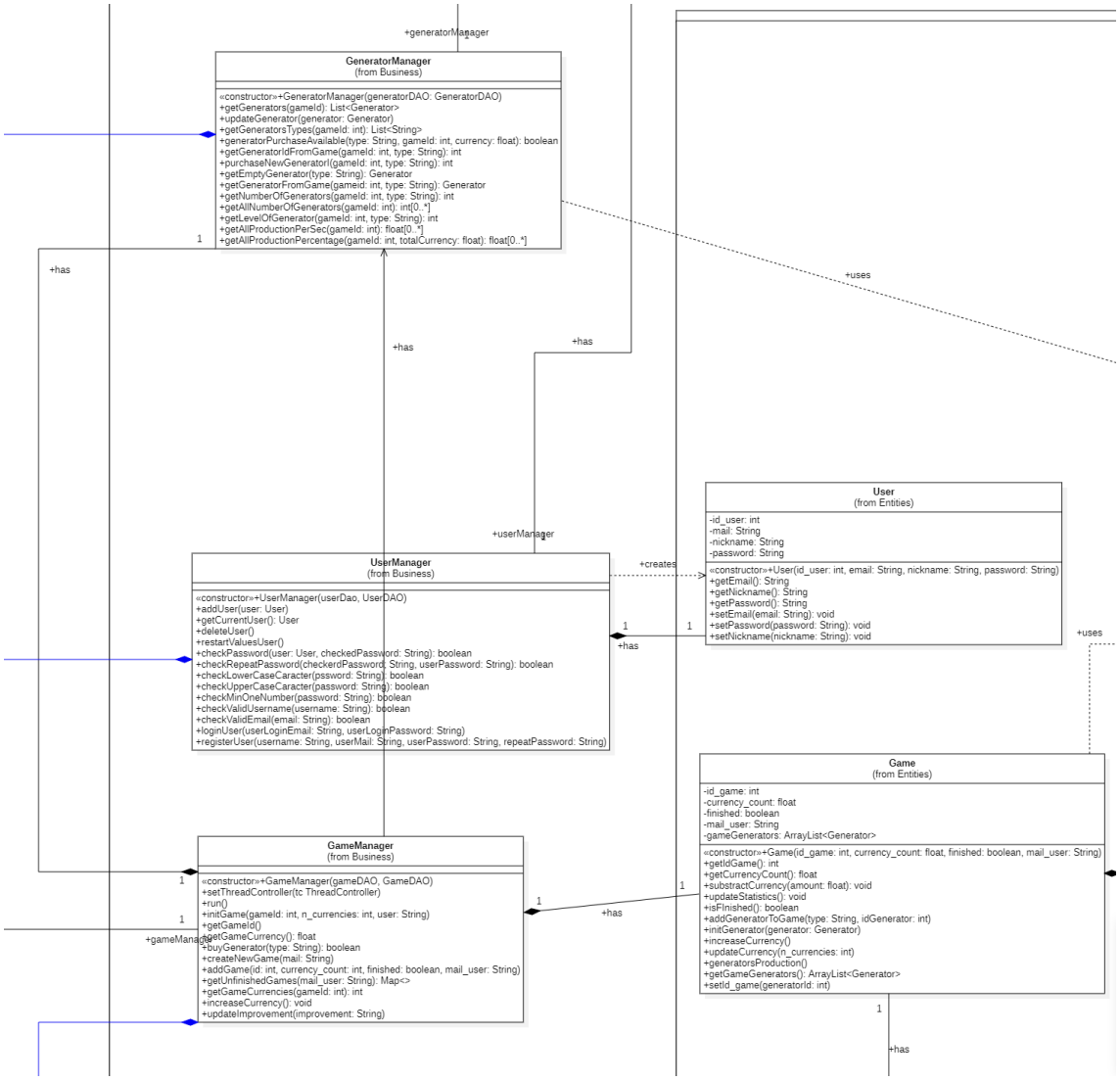


Ilustració 5 - Capa Presentation | Interfaces between Controllers and MainController

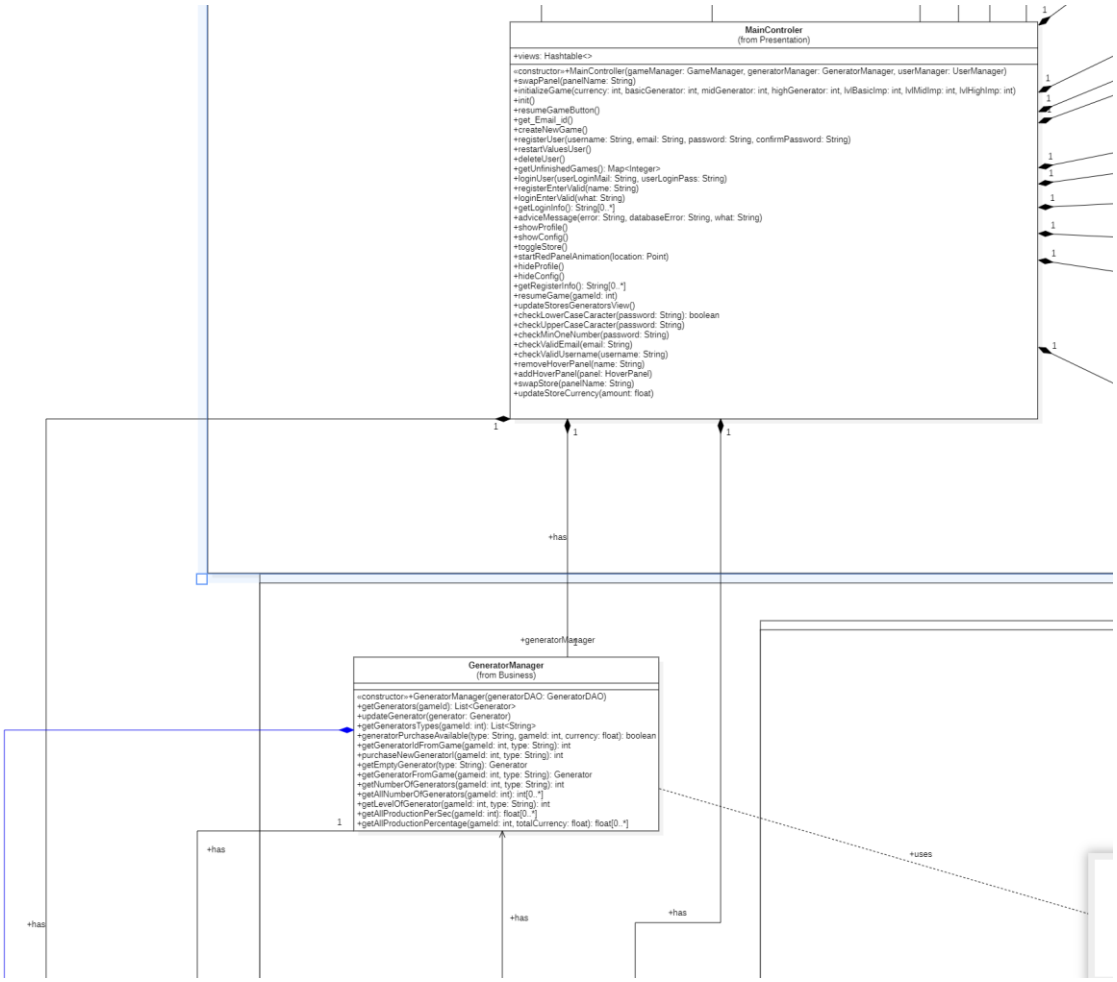


Ilustració 6 - Capa Presentation | MainController implements the Interfaces and has the Views

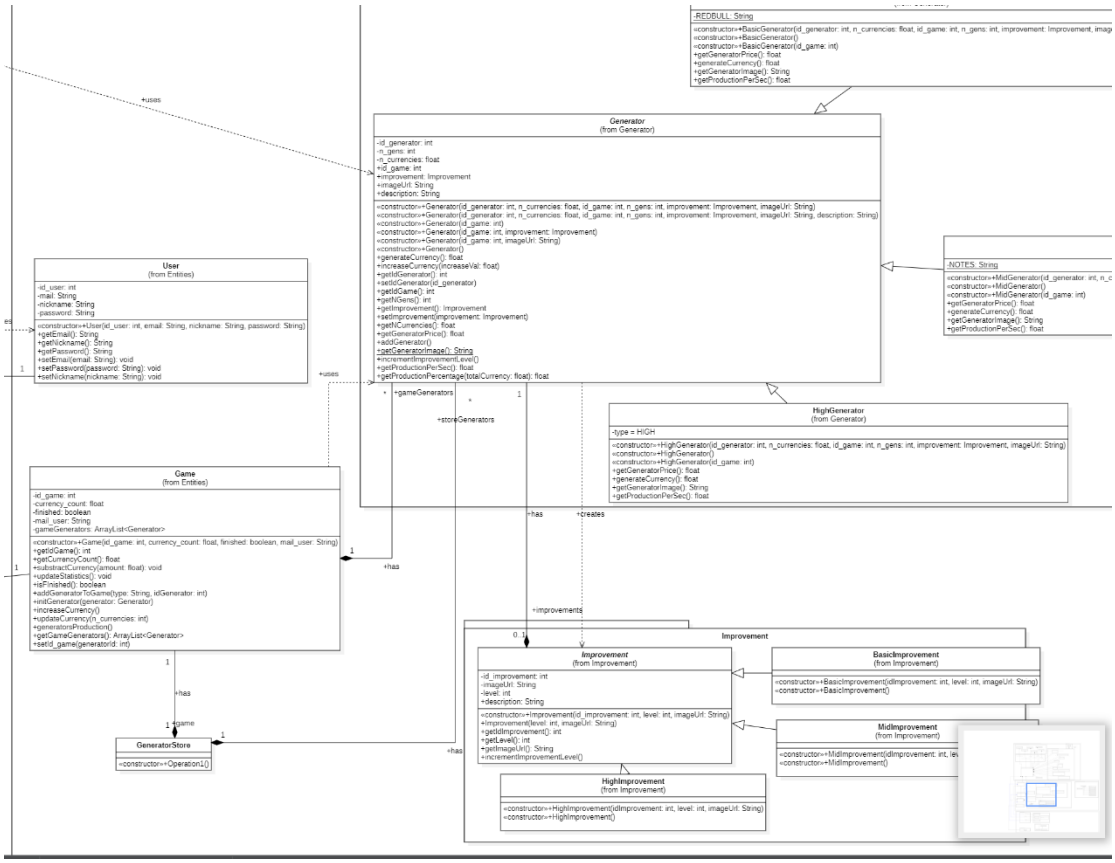
A la capa de presentació, tenim les vistes, els controladors de les vistes i el controlador principal. Aquesta capa és responsable de la interacció amb l'usuari i tradueix les accions de l'usuari en operacions comprensibles per les capes inferiors. Les vistes són les interfícies amb les quals l'usuari interactua, com ConfigView, GeneratorView, StoreView, HomeView i GameView. Cada una d'aquestes vistes té el seu propi controlador, com ConfigController, GeneratorController, StoreController, HomeController i GameController. Per centralitzar la comunicació entre la capa de presentació i la capa de negocis, hi ha un controlador principal anomenat MainController.



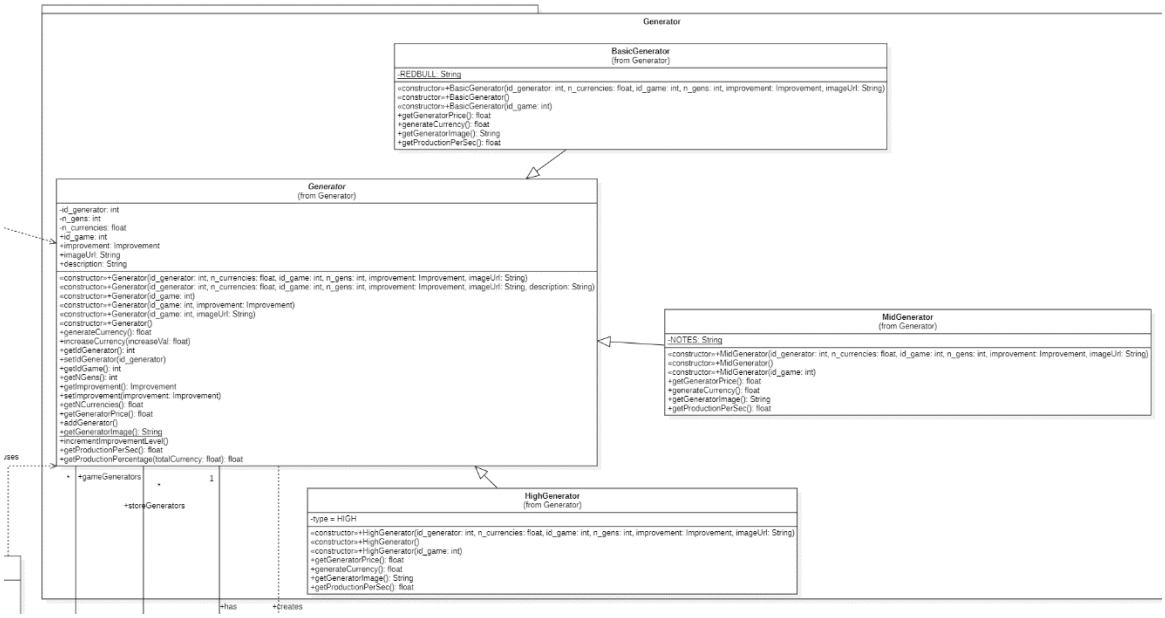
Il·lustració 7 - Capa Business | Managers i Entitats



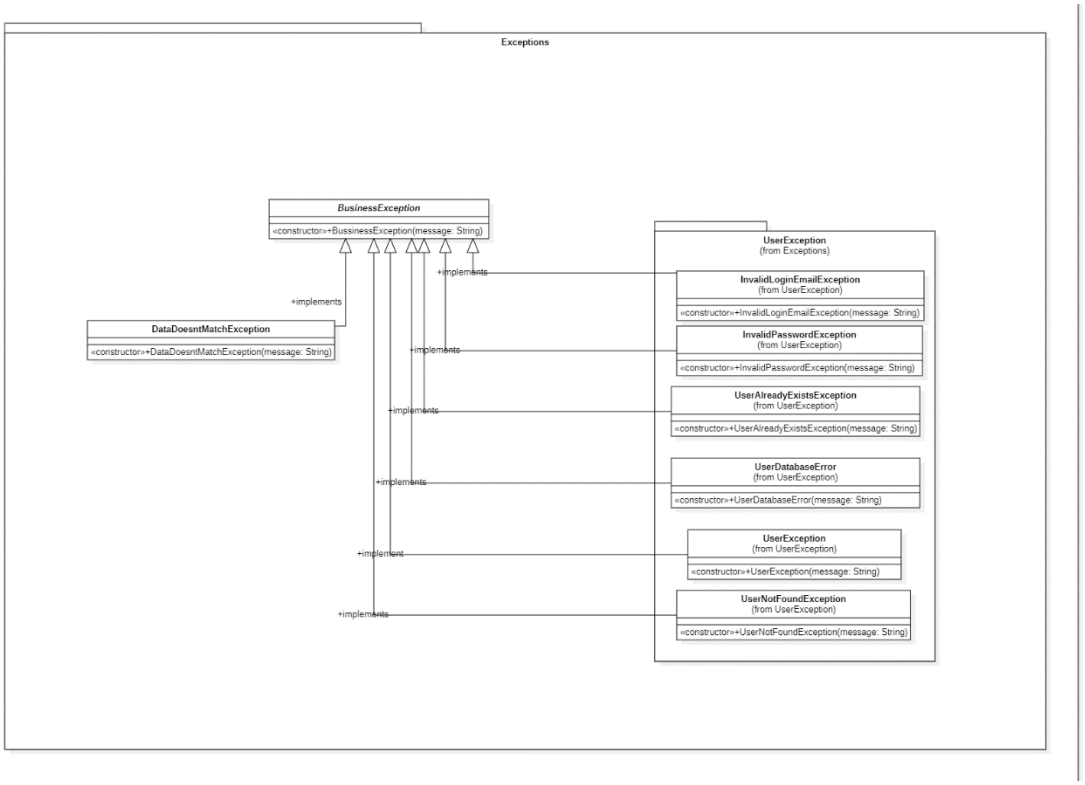
Il·lustració 8 - Capa Business | Relacions entre MainController (de capa Presentation) i les entitats de la capa de Business



Il·lustració 9 - Capa Business | Entities | Generators i Improvements



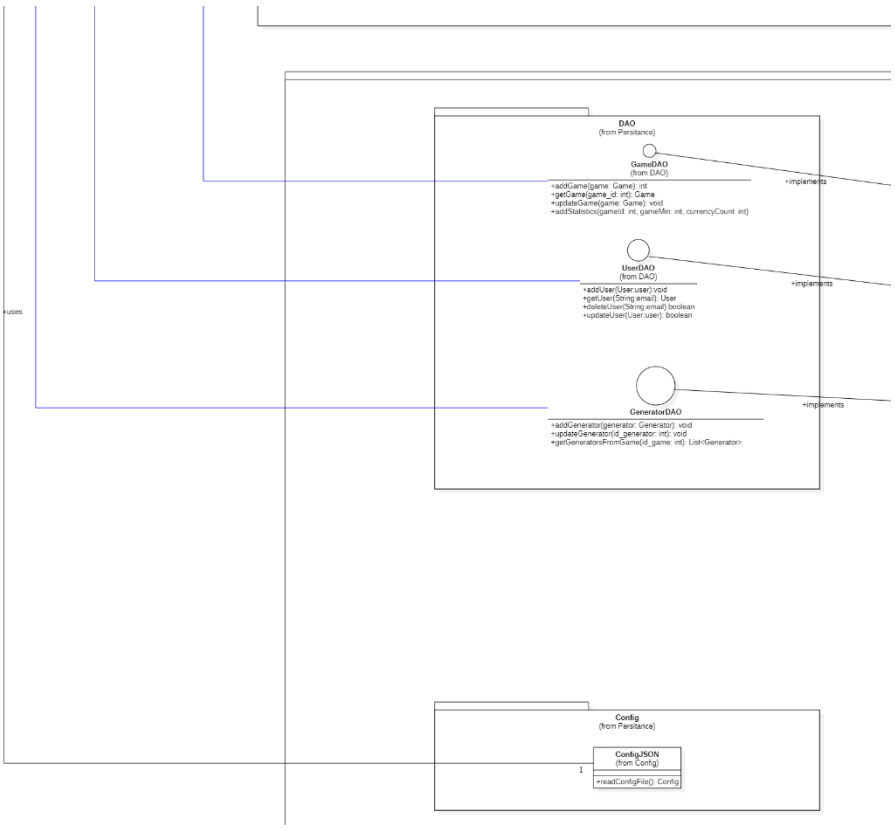
Il·lustració 10 - Capa Business | Generator



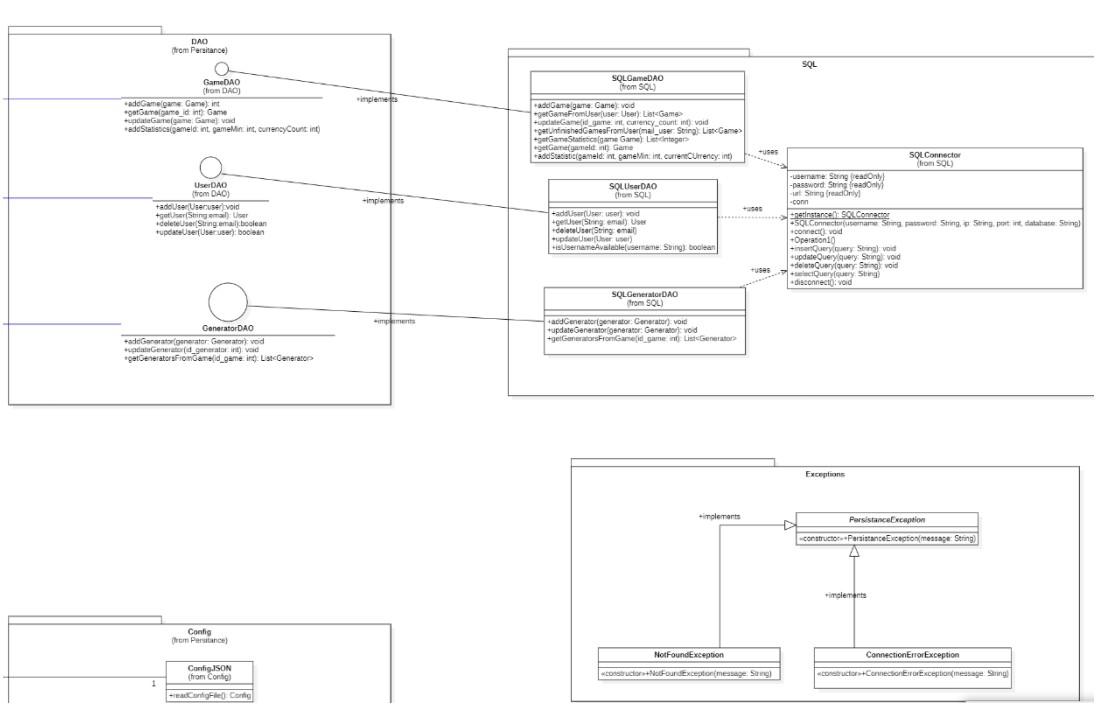
Il·lustració 11 - Capa Business | Exceptions

A la capa de negocis, tenim les entitats i les classes necessàries per implementar la lògica del joc. Això inclou entitats com Generator, Improvement, Game, User , GeneratorStore... A més, tenim els gestors UserManager, GeneratorManager i GameManager, que coordinen la lògica del joc i la interacció entre les entitats, actuen com a intermediaris entre la capa de presentació i la de persistència, i

gestionen la creació, actualització i validació de les altres entitats del projecte, com User, Game, Generator...



Il·lustració 12 - Capa Persistance | Relacions entre Persistance i Business



Il·lustració 13 - Capa Persistance: DAOs, SQL_DAOs (implementació DAOs) i Exceptions

Pel que fa a la capa de persistència, s'encarrega de la comunicació amb la base de dades. Aquesta capa inclou els DAO com UserDAO, GeneratorDAO i GameDAO, i les seves implementacions específiques per a SQL, com SQLUserDAO, SQLGameDAO i SQLGeneratorDAO. Aquesta capa assegura que les dades es gestionin correctament i es puguin persistir de manera eficient.

La comunicació entre la capa de presentació i la capa de negocis es fa a través del MainController. Per exemple, si un usuari està a la pàgina de la botiga i fa clic al botó per comprar un generador de tipus bàsic, el StoreController detecta aquesta acció i crida al mètode corresponent en la interfície del MainController. El MainController es comunica amb el gestor adequat, com el GeneratorManager, per processar la compra. Si la compra és validada, el MainController utilitza el GeneratorManager, que alhora utilitza el GeneratorDAO, implementat pel SQLGeneratorDAO, per persistir la informació a la base de dades. Després, el MainController actualitza la vista de la botiga per reflectir la compra del generador o, en cas d'error, notificar a l'usuari del problema.

Per evitar que els controladors de les vistes cridin directament al MainController, hem implementat unes interfícies. Cada controlador té una interfície amb mètodes específics que són implementats pel MainController. Això assegura una comunicació segura i controlada, seguint els principis de desacoblament i encapsulació, i permet mantenir una arquitectura flexible i escalable.

A cada capa, hem creat una carpeta específica per a les excepcions, assegurant una gestió adequada dels errors i la seva propagació controlada. Això garanteix que cada capa pugui gestionar les seves pròpies excepcions de manera ordenada.

Development methodology

Per al correcte desenvolupament de la pràctica el que vam fer va estar separar els sprints en 4 parts ven diferenciades.

En el primer sprint, ens vam centrar en analitzar els requeriments de la pràctica. Per fer-ho, vam dedicar dues classes a repassar l'enunciat per identificar tots aquells punts clau que no podíem deixar passar per alt. Un cop vam tenir clar els requeriments, el següent pas va ser crear un model preliminar del nostre diagrama UML, incloent-hi les diferents classes i la seva comunicació entre capes, utilitzant el model MVC (Model View Controller.

Un cop completada aquesta tasca inicial, vam passar a fer tant els mockups com el disseny de la base de dades. Aquestes dues parts eren essencials de completar en aquest sprint inicial, ja que constituïen la base de la pràctica. Finalment, per concloure aquest sprint, ens vam centrar en implementar la base de dades amb PhpAdmin, les consultes en Java, la classe auxiliar SQLConnector, el fitxer config.json i totes les relacions entre ells.

En el segon sprint, ens vam concentrar sobretot en la part de l'autenticació dels usuaris. Vam començar implementant el registre d'un usuari (sign up), comprovant que tots els paràmetres introduïts fossin correctes. Un cop acabat aquest punt, vam passar a implementar l'inici de sessió (sign in) per a un usuari ja registrat. Paral·lelament, vam crear les vistes corresponents per a aquesta part i les vam integrar amb la capa de negoci.

A més, també vam desenvolupar una demo de com seria la pantalla principal del joc. En aquesta demo, podíem interactuar amb l'aplicació on cada clic sumava un nou "crèdit". Tot i això, fins al sprint 3, la currency del joc (crèdits) no es guardava a persistència, és a dir, a la base de dades.

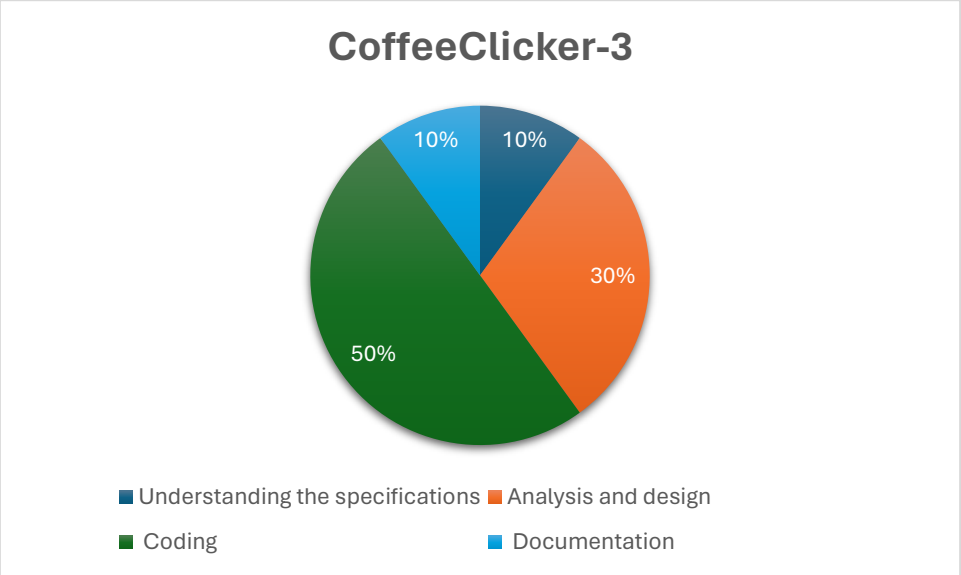
En el tercer sprint, ens vam dedicar a implementar la lògica del joc. Això incloïa iniciar una nova partida, poder continuar una partida ja existent, introduir les botigues tant de generadors com d'improvements, i permetre que un usuari autenticat pogués comprar a la botiga. També vam implementar la funcionalitat de tancar sessió (logout) i eliminar el compte (delete account), tot i que aquesta última funcionalitat la vam arrossegar fins a l'últim sprint, ja que implicava eliminar tota la informació d'un usuari i s'havia d'anar actualitzant en cada sprint.

Finalment, en l'últim sprint, ens vam centrar en implementar els threads dels generadors per generar currency automàticament. A més, vam fer diferents proves manuals per comprovar que totes les funcionalitats responien correctament, i en el cas que no vam anar solucionant cada error que ens sorgia. Per acabar, vam implementar la part de les estadístiques i vam redactar la memòria del projecte.

Time costs

Per a calcular el temps aproximat ho hem repartit en 4 parts. La primera "Understanding the specifications", vam tardar aproximament 5 hores. En aquesta etapa vam tenir que llegir diverses vegades el enunciat per entendre tots els punts bé i veure que era el que se'ns demanava. En segon lloc hi ha "Analysis and design", en aquest punt vam tenir que fer una primera vista de com serien les views mitjançant el figma. Cada vegada que fèiem una nova view teníem que anar al punt de l'enunciat on feia referència a aquella view i repassar que no ens deixéssim ningun punt important a tenir en compte, ja que sinó l'arrossegaríem cap a l'apartat de programar. Aquest mètode de dissenyar-la i veure que no ens deixàvem res va fer que tardéssim més de lo habitual, aproximament un es 15 hores. El tercer punt es el de codificar, aquest punt si que es el més dedicat en termes de hores, ja que hi ha que programar tota la lògica del joc, persistir les dades i poder mostrar-les a temps real controlant tots els possibles errors. Finalment en l'últim punt "Documentation" hem invertit 5 hores per a poder redactaren la memòria tots els apartats clars del projecte.

En conjunt fa un cúmul de 45 hores en total.



Conclusions

Aquest projecte ha estat una oportunitat valuosa per aprendre i aplicar diverses tècniques de programació i disseny de programari. Aquí es presenten algunes reflexions i aprenentatges clau:

- **Aprenentatge continu:** El desenvolupament d'aquest joc IDLE ha requerit una comprensió profunda de Java i Java Swing. Aquesta experiència ha reforçat la importància de l'aprenentatge continu en el camp de la programació.
- **Planificació i organització:** La divisió del projecte en sprints ha demostrat ser una estratègia eficaç per gestionar el temps i els recursos. Cada sprint tenia un objectiu clar, el que facilitava la concentració en tasques específiques i la resolució eficient de problemes.
- **Disseny d'interfícies d'usuari:** L'ús de Figma per dissenyar la interfície gràfica d'usuari (GUI) ha estat una part crucial del projecte. Aquesta eina ha permès visualitzar i planificar la disposició i la funcionalitat de la GUI abans de la seva implementació.
- **Gestió de dades:** La implementació de la base de dades i la gestió de les dades dels usuaris han estat una part essencial del projecte. Ha sigut el primer cop que es gestionava una base de dades relacionada amb un programa al llarg de la carrera i ha sigut un escenari que ha ajudat a entendré la seva implementació.

Es considera que s'ha seguit una bona metodologia de treball al llarg de la pràctica on s'han complert tots els terminis. S'han respectat molt les dependències de cada tasca per tal de que no es solapessin, permeten avançar d'una forma continuada i sense pausa. Aquesta ha estat una llisó apressa prèviament en altres assignatures on aquest cop s'ha aplicat des del primer moment facilitant la feina.

S'ha vist la importància de fer un disseny del programa a l'inici. Al llarg de la pràctica s'ha mantingut la base del disseny però també s'ha vist que aquest disseny es tenia que anar adaptant a les necessitats de cada moment. S'han anat fent reunions per tal de comentar aquests canvis i així estar tots al dia amb les actualitzacions del disseny.

En general, aquest projecte ha estat una oportunitat per aplicar i ampliar les habilitats de programació apreses al llarg del curs. Ha proporcionat una comprensió més profunda de com els principis de disseny i desenvolupament de programari es poden aplicar per crear una aplicació de joc interactiva i atractiva. A més, ha reforçat la importància de la planificació, l'organització i la gestió eficaç del temps en el desenvolupament de programari.

Bibliography

TutorialsPoint. *Java Programming Tutorial* [en línia]. Disponible a: https://www.tutorialspoint.com/java/index.htm [Consulta: 20 maig 2024].

Learn Java Online. *Java Tutorials* [en línia]. Disponible a: https://www.learnjavaonline.org/ [Consulta: 20 maig 2024].

MySQL. *MySQL 8.0 Reference Manual* [en línia]. Disponible a: https://dev.mysql.com/doc/ [Consulta: 20 maig 2024].

GeeksforGeeks. *Multithreading in Java* [en línia]. Disponible a: https://www.geeksforgeeks.org/multithreading-in-java/ [Consulta: 20 maig 2024].

GeeksforGeeks. *MVC Design Pattern* [en línia]. Disponible a: https://www.geeksforgeeks.org/mvc-design-pattern/ [Consulta: 20 maig 2024].

TutorialsPoint. *MVC Pattern* [en línia]. Disponible a: https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm [Consulta: 20 maig 2024].

JavaTpoint. *MVC in Java* [en línia]. Disponible a: https://www.javatpoint.com/mvc-in-java [Consulta: 20 maig 2024].